

PANDAS

What is Pandas?

Pandas is the most popular Python library for data manipulation and analysis. It provides two key data structures — Series and DataFrame — that make working with structured (tabular) data intuitive and powerful.

What Pandas Does?

Load data from CSV, Excel, SQL, JSON, and more
Clean and preprocess messy real-world data
Filter, group, and aggregate data efficiently
Merge and join multiple datasets
Perform time-series analysis
Prepare data for machine learning pipelines

Core Data Structure in pandas

Structure	Analogy	Dimensions	Example
Series	A single column with an index	1D	Temperatures across days
DataFrame	A full spreadsheet/table	2D	Student grades table

Installation And Import

```
# Installation
!pip install pandas

# Standard imports
import pandas as pd
import numpy as np
# Check version
print(pd.__version__)
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.2.2)
```

PANDAS SERIES

A one-dimensional labeled array capable of holding data of any type such as integers, strings, floats, or objects.

It contains:

Data values
Index labels

```
# CREATING A SERIES
import pandas as pd
```

```
#from List
ser=pd.Series([1,2,3,4,5])
print(ser)
```

```
0    1
1    2
2    3
3    4
4    5
dtype: int64
```

```
# Custom index
s2 = pd.Series([100, 200, 300], index=['a', 'b', 'c'])
print(s2['b'])
```

```
200
```

```
# From dictionary
d = {'vamshi': 90, 'harsha': 85, 'rani': 78}
marks = pd.Series(d)
print(marks)
```

```
vamshi    90
harsha    85
rani      78
dtype: int64
```

Series Operations

```
s = pd.Series([1, 2, 3, 4, 5])
```

```
# Arithmetic (element-wise)
s * 2
s+10
```

```
    0
0   11
1   12
2   13
3   14
4   15
```

```
dtype: int64
```

```
# Statistics
s.mean()
s.sum()
s.max()
s.describe()
```

```
    0
count  5.000000
mean   3.000000
std    1.581139
min    1.000000
25%    2.000000
50%    3.000000
75%    4.000000
max    5.000000
```

```
dtype: float64
```

```
# Boolean indexing
s[s > 3]
```

```
    0
3   4
4   5
```

```
dtype: int64
```

```
# Vectorized string operations
names = pd.Series(['Vamsi', 'harsha', 'Rani'])
#names.str.upper()
#names.str.len()
names.str.startswith('h')
```

```

      0
0  False
1   True
2  False

dtype: bool

```

PANDAS DATAFRAME A two-dimensional labeled data structure consisting of rows and columns, capable of storing different types of data.

It is similar to:

an Excel table a SQL table or a spreadsheet

CREATING A DATAFRAME

```

# From dictionary
data = {
    'Name':  ['ravi', 'hari', 'siri', 'raju'],
    'Age':   [23, 25, 22, 28],
    'Score': [88, 75, 92, 81],
    'Grade': ['A', 'B', 'A', 'A']
}
a = pd.DataFrame(data)
print(a)

```

<built-in function a>

```

# From list of dicts
rows = [{'x': 1, 'y': 2}, {'x': 3, 'y': 4}]
b = pd.DataFrame(rows)
print(b)

```

```

   x  y
0  1  2
1  3  4

```

```

# From NumPy array
arr = np.random.rand(3, 4)
c = pd.DataFrame(arr, columns=['A', 'B', 'C', 'D'], index=['x', 'y', 'z'])
print(c)

```

```

      A         B         C         D
x  0.715538  0.103511  0.373666  0.652166
y  0.737853  0.417775  0.796219  0.037396
z  0.523697  0.704981  0.195852  0.515720

```

```

# Exploring Data Frame
data = {
    'Name':  ['ravi', 'hari', 'siri', 'raju'],
    'Age':   [23, 25, 22, 28],
    'Score': [88, 75, 92, 81],
    'Grade': ['A', 'B', 'A', 'A']
}
d = pd.DataFrame(data)

```

```

print(d.head(3))      # First n rows (default: 5)
d.tail(3)             # Last 3 rows
d.shape               # (4, 4) - rows, columns
d.info()              # Column names, dtypes, non-null counts
d.describe()          # Statistical summary of numeric columns
d.columns             # Index(['Name', 'Age', 'Score', 'Grade'])
d.index               # RangeIndex(start=0, stop=4, step=1)
d.dtypes              # dtype of each column
d.nunique()           # Count of unique values per column
d.isnull().sum()      # Count of missing values per column

```

```

      Name  Age  Score  Grade
0   ravi   23    88      A
1   hari   25    75      B
2   siri   22    92      A
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  ---
0   Name    4 non-null        object
1   Age     4 non-null        int64
2   Score   4 non-null        int64
3   Grade   4 non-null        object
dtypes: int64(2), object(2)
memory usage: 260.0+ bytes

```

```

      0
-----
Name  0
Age   0
Score 0
Grade 0

dtype: int64

```

Selecting and Filtering Data

```

#Selecting Columns
# Single column → returns Series

d['Name']
d.Name
# Multiple columns → returns DataFrame
d[['Name', 'Score']]

```

	Name	Score
0	ravi	88
1	hari	75
2	siri	92
3	raju	81

✓ Selecting Rows — .loc[] and .iloc[]

.loc[] → Label-Based Selection

.loc[] selects data using:

row labels (index names)

column names

.iloc[] → Position-Based Selection

.iloc[] selects data using:

row numbers

column numbers

Index starts from 0.

```

# .loc[] — label-based (uses index labels)
d.loc[0]           # Row with index 0
d.loc[0:2]         # Rows 0, 1, 2 (inclusive!)
d.loc[0:2, 'Name':'Score'] # Rows AND columns by label

```

	Name	Age	Score
0	ravi	23	88
1	hari	25	75
2	siri	22	92

```
# .iloc[] – integer-based (like NumPy slicing)
d.iloc[0]           # First row
d.iloc[0:2]         # Rows 0, 1 (exclusive stop!)
d.iloc[0:2, 0:2]    # Rows 0-1, Columns 0-1
d.iloc[-1]         # Last row
```

	3
Name	raju
Age	28
Score	81
Grade	A

dtype: object

```
# Single value
d.loc[0, 'Name']
d.iloc[0, 0]
```

'ravi'

Filtering Rows and Columns

```
# Single condition
d[d['Score'] > 80]
```

	Name	Age	Score	Grade
0	ravi	23	88	A
2	siri	22	92	A
3	raju	28	81	A

```
# Multiple conditions: use & (and) | (or) ~ (not)
d[(d['Score'] > 80) & (d['Grade'] == 'A')]
d[(d['Age'] < 24) | (d['Score'] > 90)]
```

	Name	Age	Score	Grade
0	ravi	23	88	A
2	siri	22	92	A

```
# .query() – string-based filtering (cleaner syntax)
d.query('Score > 80 and Age < 25')
d.query('Grade == "A"')
```

	Name	Age	Score	Grade
0	ravi	23	88	A
2	siri	22	92	A
3	raju	28	81	A

```
# isin() – filter on multiple values
d[d['Name'].isin(['raju', 'ravi'])]
```

	Name	Age	Score	Grade
0	ravi	23	88	A
3	raju	28	81	A

ADDING AND MODIFYING

```
# Add a new column
d['Bonus'] = d['Score'] * 0.1
d['Pass'] = d['Score'] >= 80
print(d)
```

	Name	Age	Score	Grade	Bonus	Pass
0	ravi	23	88	A	8.8	True
1	hari	25	75	B	7.5	False
2	siri	22	92	A	9.2	True
3	raju	28	81	A	8.1	True

```
# Modify existing column
d['Score'] = d['Score'] + 5
```

```
def grade(score):
    if score >= 90:
        return 'A'
    elif score >= 80:
        return 'B'
    else:
        return 'C'

# apply() with function
d['Grade_Letter'] = d['Score'].apply(grade)

print(d)
```

	Name	Age	Score	Grade	Bonus	Pass	Grade_Letter
0	ravi	23	93	A	8.8	True	A
1	hari	25	80	B	7.5	False	B
2	siri	22	97	A	9.2	True	A
3	raju	28	86	A	8.1	True	B

```
# map() - map Series values to new values
grade_map = {'A': 4.0, 'B': 3.0, 'C': 2.0}
d['GPA'] = d['Grade'].map(grade_map)

print(d)
```

	Name	Age	Score	Grade	Bonus	Pass	Grade_Letter	GPA
0	ravi	23	93	A	8.8	True	A	4.0
1	hari	25	80	B	7.5	False	B	3.0
2	siri	22	97	A	9.2	True	A	4.0
3	raju	28	86	A	8.1	True	B	4.0

```
# Drop columns
d.drop('Bonus', axis=1, inplace=True)
d.drop(columns=['Pass', 'GPA'], inplace=True)
# 'Bonus' → column name to remove
# axis=1 → means column (not row)
# inplace=True → modifies the original DataFrame d directly#
```

```
# Rename columns
d.rename(columns={'Score': 'Marks', 'Name': 'Student'}, inplace=True)
print(d)
```

	Student	Age	Marks	Grade	Grade_Letter
0	ravi	23	93	A	A
1	hari	25	80	B	B
2	siri	22	97	A	A
3	raju	28	86	A	B

HANDLING MISSING DATA

```
import numpy as np
# Detect missing values
d.isnull() # Boolean DataFrame
d.isnull().sum() # Missing count per column#
```

```
d.notnull()          # Opposite of isnull
```

	Student	Age	Marks	Grade	Grade_Letter
0	True	True	True	True	True
1	True	True	True	True	True
2	True	True	True	True	True
3	True	True	True	True	True

```
# Drop rows/columns with NaN
d.dropna()          # Drop rows with any NaN
d.dropna(axis=1)     # Drop columns with any NaN
d.dropna(thresh=3)   # Keep rows with >= 3 non-NaN
```

	Student	Age	Marks	Grade	Grade_Letter
0	ravi	23	93	A	A
1	hari	25	80	B	B
2	siri	22	97	A	A
3	raju	28	86	A	B

```
# Fill missing values
d.fillna(0)          # Replace all NaN with 0
d['Age'].fillna("not mentioned", inplace=True) # Mean imputation
d.fillna(method='ffill') # Forward-fill (carry forward)
d.fillna(method='bfill') # Backward-fill
print(d)
```

	Student	Age	Marks	Grade	Grade_Letter
0	ravi	23	93	A	A
1	hari	25	80	B	B
2	siri	22	97	A	A
3	raju	28	86	A	B

/tmp/ipykernel_3643/1438260921.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through `inplace` and it has no copy. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are performing the operation is a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col].method(value, inplace=True)

```
d['Age'].fillna("not mentioned", inplace=True) # Mean imputation
/tmp/ipykernel_3643/1438260921.py:4: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version.
d.fillna(method='ffill') # Forward-fill (carry forward)
/tmp/ipykernel_3643/1438260921.py:5: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version.
d.fillna(method='bfill') # Backward-fill
```

```
# Replace specific values
d.replace(93, np.nan, inplace=True)
```

```
print(d)
```

	Student	Age	Marks	Grade	Grade_Letter
0	ravi	23	NaN	A	A
1	hari	25	80.0	B	B
2	siri	22	97.0	A	A
3	raju	28	86.0	A	B

Sorting And Ranking

```
# Sort by single column
d.sort_values('Marks', ascending=True)
```

	Student	Age	Marks	Grade	Grade_Letter
1	hari	25	80.0	B	B
3	raju	28	86.0	A	B
2	siri	22	97.0	A	A
0	ravi	23	NaN	A	A

```
# Sort by multiple columns
d.sort_values(['Grade', 'Marks'], ascending=[True, False])
```

	Student	Age	Marks	Grade	Grade_Letter
2	siri	22	97.0	A	A
3	raju	28	86.0	A	B
0	ravi	23	NaN	A	A
1	hari	25	80.0	B	B

```
# Sort by index
d.sort_index(ascending=True)
```

	Student	Age	Marks	Grade	Grade_Letter
0	ravi	23	NaN	A	A
1	hari	25	80.0	B	B
2	siri	22	97.0	A	A
3	raju	28	86.0	A	B

```
# Get top N rows
d.nlargest(3, 'Marks')    # Top 3 scores
d.nsmallest(2, 'Age')     # 2 youngest
```

	Student	Age	Marks	Grade	Grade_Letter
2	siri	22	97.0	A	A
0	ravi	23	NaN	A	A

GroubBy and Aggregation `groupby()` splits data into groups, applies a function, and combines the results — the 'split-apply-combine' pattern.

```
data = {
    'Department': ['IT', 'HR', 'IT', 'HR', 'IT'],
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'Salary': [70000, 50000, 80000, 55000, 75000],
    'YearsExp': [3, 5, 7, 2, 4]
}
df = pd.DataFrame(data)
```

```
# Basic groupby with single aggregate
df.groupby('Department')['Salary'].max()
```

	Salary
Department	
HR	55000
IT	80000

dtype: int64

```
# Multiple aggregations
df.groupby('Department').agg({
    'Salary': ['mean', 'max', 'min'],
    'YearsExp': 'sum'
})
```


	Salary		YearsExp		
	mean	max	min	sum	
Department					
HR	52500.0	55000	50000		7
IT	75000.0	80000	70000		14

```
# agg with custom functions
df.groupby('Department')['Salary'].agg([
    ('Average', 'mean'),
    ('Highest', 'max'),
    ('Count', 'count')
])
```

	Average	Highest	Count	
Department				
HR	52500.0	55000	2	
IT	75000.0	80000	3	

```
# Group size
df.groupby('Department').size()
```

	0
Department	
HR	2
IT	3

dtype: int64

Merging and Joining DataFrames

```
employees = pd.DataFrame({
    'emp_id': [1, 2, 3, 4],
    'name': ['Alice', 'Bob', 'Charlie', 'Diana'],
    'dept_id': [10, 20, 10, 30]
})

departments = pd.DataFrame({
    'dept_id': [10, 20],
    'dept_name': ['Engineering', 'HR']
})
```

```
# INNER JOIN – only matching rows
pd.merge(employees, departments, on='dept_id', how='inner')
```

	emp_id	name	dept_id	dept_name
0	1	Alice	10	Engineering
1	2	Bob	20	HR
2	3	Charlie	10	Engineering

```
# LEFT JOIN – all from left, matching from right
pd.merge(employees, departments, on='dept_id', how='left')
```

	emp_id	name	dept_id	dept_name
0	1	Alice	10	Engineering
1	2	Bob	20	HR
2	3	Charlie	10	Engineering
3	4	Diana	30	NaN

```
# RIGHT JOIN - all from right
pd.merge(employees, departments, on='dept_id', how='right')
```

	emp_id	name	dept_id	dept_name
0	1	Alice	10	Engineering
1	3	Charlie	10	Engineering
2	2	Bob	20	HR

```
# OUTER JOIN - all rows from both
pd.merge(employees, departments, on='dept_id', how='outer')
```

	emp_id	name	dept_id	dept_name
0	1	Alice	10	Engineering
1	3	Charlie	10	Engineering
2	2	Bob	20	HR
3	4	Diana	30	NaN

Join Type	Keeps	NaN Introduced?
inner	Only rows with matching keys in BOTH tables	No
left	All rows from LEFT table	Yes (for missing right values)
right	All rows from RIGHT table	Yes (for missing left values)
outer	All rows from BOTH tables	Yes (for non-matching rows)

Pandas — Practice Problems

Use the following dataset for Q1–Q8:

```
import pandas as pd
```

```
data = { 'StudentID': [101, 102, 103, 104, 105, 106],
```

```
  'Name':   ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank'],
  'Dept':   ['CS', 'IT', 'CS', 'DS', 'IT', 'CS'],

  'Marks':  [88, 75, 92, None, 68, 95],

  'Attend': [90, 85, 78, 92, None, 88]
```

```
}
```

```
df = pd.DataFrame(data)
```

Q1. Print the shape, dtypes, and first 3 rows of the DataFrame.

Q2. Identify and fill missing values: fill Marks with the mean of Marks, and Attend with the median of Attend. 💡 Use `.fillna()` with `.mean()` and `.median()`

Q3. Add a new column 'Result' that is 'Pass' if Marks $\geq$ 75 and Attend $\geq$ 80, else 'Fail'. 💡 Use `np.where()` or `apply()`

Q4. Filter and display only CS department students with Marks $>$ 85.

Q5. Sort the DataFrame by Marks in descending order and display the top 3 students.

Q6. Find the average Marks and average Attendance grouped by Department. 💡 Use `groupby().agg()`

Q7. Create a second DataFrame with columns StudentID and Email. Merge it with the original df using a left join on StudentID. 💡 Use `pd.merge()` with `how='left'`

Q8. Create a pivot table showing average Marks per Department. 💡 Use `df.pivot_table()`

